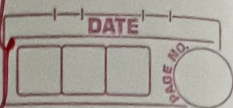


**
* } by default, all classes in Java
are implicitly inherited from the
Object class



(Also known as Cosmic

* Object Class in Java $\Rightarrow$ Super class)

- i) The Object class in Java is the root class of the Java class hierarchy.
- ii) This means that all the Java classes have access to the methods defined in the objects.
- iii) It is part of the java.lang package, and no explicit import is required to use it.

IMP ^{some} Methods of Object Class $\rightarrow$

① public boolean equals (Object obj) $\Rightarrow$

i) Compares the current object with the specified object for equality.

ii) The default implementation compares object references.

iii) Typically overridden in classes to define custom equality logic.

② public String toString() $\Rightarrow$

i) Returns a string representation of the object

ii) The default implementation returns the class name followed by object hash code.

iii) Often overridden to provide meaningful output.

③ public int hashCode () $\Rightarrow$

i) Return the hash code for the Object.

ii) Default implementation returns a hash code based on the object's memory

Code $\rightarrow$

```
class Person {  
    private String name;  
    private int age;
```

```
    public Person (String name, int age) {  
        this.name = name;  
        this.age = age;
```

```
    }
```

```
    // No overridden methods
```

```
}
```

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        person p1 = new person ("Ankit", 18);
```

```
        person p2 = new person ("Ankit", 18);
```

```
        sysout (p1.equals(p2)); // false
```

```
        sysout (p1.toString()); // Person @ 7a81197d hashcode
```

```
        sysout (p2.toString()); // Person @ <hashcode>
```

```
        sysout (p1.hashCode()); // unique hashcode for p1
```

```
        sysout (p2.hashCode()); // 11 P2
```

p1.getClass().getName();
me());

// gets class name
output $\rightarrow$ Person

* Wrapper class $\Rightarrow$

A wrapper class provides an object representation for primitive datatypes. Primitive types (like int, double, char, etc) are not objects.

(That's why java is not considered as 100% object oriented language because it allows for primitive data types like 'int' and 'boolean', etc which are not objects, and also supports static methods and variables that are not directly tied to an object instance.) Why java is not 100% object oriented language.

So wrapper classes "wrap" these primitive types into objects, enabling them to be used in contexts where an objects are required. such as with collections like ArrayList.

Primitive Type & Their Wrapper class

Primitive Type		Wrapper class
i) byte	$\longrightarrow$	Byte
ii) short	$\longrightarrow$	Short
iii) int	$\longrightarrow$	Integer
iv) long	$\longrightarrow$	Long
v) float	$\longrightarrow$	Float
vi) double	$\longrightarrow$	Double
vii) char	$\longrightarrow$	Character
viii) boolean.	$\longrightarrow$	Boolean

Note →

i) Collections like ArrayList and HashMap work only with objects, not primitive.
(You can store an Integer in an ArrayList but not an int.)

ii) Wrapper classes provide useful methods parsing, valueOf, conversion, etc.

iii) Java allows automatic conversion between primitive types and their corresponding wrapper class. (i.e. Autoboxing, Unboxing)

① Autoboxing ⇒

Automatic conversion of a primitive type to its wrapper class.

```
int num = 10;
Integer obj = num; // Autoboxing
```

② Unboxing ⇒

Automatic conversion of a wrapper class object to its corresponding primitive type.

```
Integer obj = 20;
int num = obj; // Unboxing
```


* Important Wrapper classes Methods $\Rightarrow$

① Parse (String s) $\rightarrow$

Converts a string to the corresponding primitive type.

```
int num = Integer.parseInt("123");
double d = Double.parseDouble("3.14");
```

② ValueOf (String s) $\rightarrow$

Converts a string to a wrapper ~~object~~ ^{object}

```
Integer num = Integer.valueOf("123");
```

③ toString() $\rightarrow$

Converts a wrapper object to its string representation.

```
Integer num = 1000;
String str = num.toString();
```

④ Value() $\rightarrow$

Converts a wrapper object to its corresponding primitive value

```
Integer obj = 50;
int num = obj.intValue();
```


* Anonymous blocks $\Rightarrow$

In Java, an anonymous block is a block of code without a name that is executed when the class is loaded. It is also known as an instance initializer block.

Syntax $\Rightarrow$

```
{  
    // code to be executed  
}
```

* Inner class in Java $\Rightarrow$

An Inner class in Java is a class that is declared inside another class.

Code $\Rightarrow$

```
class Outer {
```

```
    private String message = "Hello im from outer  
                                class";
```

```
    class Inner {
```

```
        void display() {
```

```
            syso(message); // Accessing private  
                             member of outer  
                             class  
        }
```


Anonymous inner class are typically used in

- i) Extending a class (without creating subclasses)
- ii) Implementing an Interface

DATE:

PAGE NO.:

* Anonymous Inner class

Anonymous inner class is a class without a name that is created and used at the same time (created by new keyword)

code $\Rightarrow$

** used for single use
Implementation.

Interface
only provide method declaration, without defining

Class A (can be declare inside a method, constructor, or another class)

```

public void show()
{
    syso("In A show");
}

public static void main (String[] args)
{
    A ref = new A()
    {
        @Override
        public void show()
        {
            syso("In Anonymous class");
        }
    };
    ref.show();
}

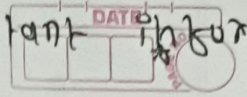
```

JVM cannot allocate memory for an instance (Object) of the interface

Interface do not have concrete implementation

Interface cannot instantiated directly because it only define method signature (no definition) But using Anonymous inner class it looks like object is created but in reality, Java is creating an instance of an anonymous class that implements the interface

Annotations helps makes code cleaner and safer by giving developers & tools important information without changing the logic }



* Annotations in Java $\Rightarrow$

Annotations are special tags or markers that provides metadata (Extra Information) about a program.

They don't change the behaviour of the program, It is used to give the compiler some instructions without affecting the actual code.

Syntax $\Rightarrow$ Code Example

```
@Override // Override annotation
public String toString() {
    return "My object";
}
```

Common Annotations

- ① @Override $\rightarrow$ Tells the compiler that a method is overriding a method from superclass.
- ② Deprecated $\rightarrow$ Marks a method or class as outdated, & the compiler warns us.
- ③ @SuppressWarnings $\rightarrow$ Instructs the compiler to suppress specific warnings for certain code sections.

* Enum (Enumeration) $\Rightarrow$

An Enum (short for "enumeration") in Java is a special data type used to define a fixed set of constant values.

Enums are useful when a variable should only take one of a predefined set of values.

Enums are declared using the enum keyword and they can contain constants, constructors, methods, etc.

Common methods in Enum $\rightarrow$

① values() $\rightarrow$

Returns an array of all enum constants

② ordinal() $\rightarrow$

Returns the index of an enum constant (starts from 0).

③ valueOf(String name) $\rightarrow$

Converts a string to an enum constant.

Example (code) $\Rightarrow$

```
enum Day {  
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY;  
}  
  
public class EnumExample {  
    public static void main (String[] args) {  
        Day today = Day.SUNDAY;  
        System.out.println("Today is : " + today);  
    }  
}
```

output $\rightarrow$

Today is : SUNDAY

Note $\rightarrow$

i) Enum extends java.lang.Enum by default (cannot extend another class)

ii) Enums can implement interfaces

iii) Enum can have constructors, fields, and methods but cannot be instantiated directly (new keyword is not allowed)

* Types of Interface =>

- ① Normal Interface
- ② Marker Interface
- ③ Functional Interface (SAM)

① Normal Interface :->

This is the most common type of interface, where you define a contract for classes to implement

It has abstract methods (method without a body) and default methods (methods with a body)

Code ->

```

interface Animal {
    void sound(); // abstract method
    default void sleep() { // Only default method can
        system.out.println("Sleeping"); // be write
    } // no normal method allowed with a body
}
    
```


*** Stream API (added in Java 8) that helps you to process data (like form a list, set, or array) easily & quickly most commonly used for filtering, sorting, changing values, counting with minimal boilerplate code

Commonly used Methods
(1) filter()
(2) map()
(3) sorted()
(4) collect
(5) count

② Marker Interface :- →

A marker interface does not have any methods. It is used to mark a class with particular property or behaviour.

Serializable and Cloneable are marker interfaces in Java.

Code →

```
interface Serializable {
```

```
// No methods
```

```
}
```

③ Functional Interface :- →

A functional interface has only one abstract method.

It can have multiple default or static methods.

This is used in lambda expressions and method reference.

It is marked with the @FunctionalInterface annotation, though the annotation is optional.

*** What is Optional class in Java & →
used to represent the presence or absence of a value.
It helps to avoid the NullPointerException
(Durgesh)



Code →

```
@FunctionalInterface // Annotation
interface Calculator {
    int add(int a, int b); // Abstract method
    default int subtract(int a, int b) { // Default method
        return a - b;
    }
}
```

* What is default Methods in Java ?

In Java, default method (also called defender methods)

They allow interfaces to have methods with implementations without breaking existing classes that implement the interface.

Note →

- i) Defined in an interface using default keyword
- ii) Provides a default implementation that can be overridden by implementing classes.

(parameter) $\rightarrow$ method argument (parameter)

($\rightarrow$) $\rightarrow$ Lambda arrow

{body} $\rightarrow$ method body (single or multiple)

* Lambda Expression (is an anonymous comparator and comparable $\Rightarrow$ function)

i) Lambda Expression is used to implement by using functional Interface

ii) A functional interface is a interface that contain only one abstract method

iii) @FunctionalInterface (anotation) is optional but helps to know that it is functional Interface.

Example $\Rightarrow$ (Syntax $\rightarrow$ (parameter) $\rightarrow$ {body})

@FunctionalInterface

interface MyInterface {

void showMessage(String message);
}

public class LambdaExample {

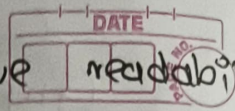
public static void main(String[] args) {

MyInterface myInterface = (message) $\rightarrow$
System.out.println("Message" + message);

// lambda Expression implement interface

myInterface.showMessage("Hello, Lambda!");
}

§ Lambda Expression can only be used to
Implement functional Interface
It reduce boilerplate code & improve readability



* Examples $\Rightarrow$

① Lambda Expression with a single Parameter

```
interface Greeting {  
    void sayHello (String name);  
}  
  
public class LambdaExample {  
    public static void main (String args[]) {  
        Greeting greet = (name)  $\rightarrow$  System.out.println  
            ("Hello" + name);  
        greet.sayHello ("Ankit");  
    }  
}
```

② Lambda Expression with Multiple Parameters

```
interface MathOperation {  
    int add (int a, int b);  
}  
  
public class LambdaExp {  
    public static void main (String[] args) {  
        MathOperation addition = (a,b)  $\rightarrow$  a + b;  
        Sys0 ("Addition" + addition.add (5, 10));  
    }  
}
```


Java 8 was officially released on March 18 2024.
It introduced major features like
Lambda Expressions, Streams API, functional Interface,
default Methods, Optional, and new Date and
Static Method in interface
Time API }

③ Lambda Expression with no parameters

```
interface Message {  
    void show();  
}  
  
public class LambdaExp {  
    public static void main (String[] args) {  
        Message m = () -> System.out.println  
            ("Hello, Lambda");  
        m.show();  
    }  
}
```

④ Lambda Expression with multi line

```
interface MathOp {  
    int multi (int a, int b);  
}  
  
public class LambdaExp {  
    public static void main (String[] args) {  
        MathOp multiply = (a,b) -> {  
            System.out.println ("Multiplying" + a + "and" + b);  
            int result = a * b;  
            return result;  
        };  
        System.out.println ("Result : " + multiply.multi(5,10));  
    }  
}
```


* Containment :->

In java, containment is an important concept in object-oriented programming (OOPS). It also known as "has-a" relationship.

(meaning one class contains a reference to another class) This allows code reusability and modular design.

Definition :->

Containment refers to the process where an object of the one class is used within another class.

This is different from inheritance (which is an "is-a" relationship).

Example.

A car has a Engine. Here, car class contains an instance of Engine class.

Why use Containment-?

- i> Promotes code reusability.
- ii> Helps in modular and manageable designs
- iii> Encourages composition over inheritance.

* Types of Containment in Java :->

① Composition (Strong Containment) :->

The contained object cannot exist independently of the container.

If the container is destroyed, the contained object is also destroyed.

Example :->

```
class Engine {
    void start() {
        System.out.println("Engine started.");
    }
}

class Car {
    private Engine engine = new Engine();
    // composition.

    void startCar() {
        engine.start();
        System.out.println("Car started.");
    }
}
```


② Aggregation (Weak Containment) :->

The contained object can exist independently of the container.

If the container is destroyed, the contained object can still exist.

Example :->

```
class Department {
    String name;
    Department(String name) {
        this.name = name;
    }
}
```

```
class Student {
```

```
    String name;
    Department dept;
```

```
    Student(String name, Department dept) {
```

```
        this.name = name;
        this.dept = dept;
```

```
    }
```

```
}
```

Here, Department can exist independently of Student.